

```
"""
```

```
Model Training, Validation &  
Deployment Prep  
Week 3 Task - Yuva Intern (Data  
Science Project Coordinator)  
Author: Dharanya
```

```
Implements the execution-phase model  
development workflow described in the  
Week 3 Execution & Implementation  
Plan: model selection, training,  
validation, and conceptual deployment  
readiness checks.
```

```
"""
```

```
import pandas as pd  
from sklearn.model_selection import  
train_test_split, cross_val_score  
from sklearn.linear_model import  
LogisticRegression  
from sklearn.ensemble import  
RandomForestClassifier  
from sklearn.metrics import  
accuracy_score, precision_score,  
recall_score, f1_score  
import joblib
```

```

# -----
-----
---
# Step 1: Model Selection
# -----
-----
---
def get_candidate_models() -> dict:
    """Return candidate models to
    compare, per the model-selection
    strategy."""
    return {
        "LogisticRegression":
        LogisticRegression(max_iter=1000,
        random_state=42),
        "RandomForest":
        RandomForestClassifier(n_estimators=2
        00, random_state=42),
    }

# -----
-----
---
# Step 2: Training
# -----
-----

```

```

---
def train_and_compare(models: dict,
X_train, y_train, cv: int = 5) ->
dict:
    """Train each candidate model and
score it via cross-validation."""
    results = {}
    for name, model in
models.items():
        scores =
cross_val_score(model, X_train,
y_train, cv=cv, scoring="f1")
        model.fit(X_train, y_train)
        results[name] = {"model":
model, "cv_f1_mean": scores.mean()}
        print(f"{name}: mean CV F1 =
{scores.mean():.4f}")
    return results

# -----
-----

---
# Step 3: Validation
# -----
-----

---
def validate_model(model, X_test,
y_test) -> dict:

```

```

        """Evaluate the chosen model on
the held-out validation/test set."""
        preds = model.predict(X_test)
        metrics = {
            "accuracy":
accuracy_score(y_test, preds),
            "precision":
precision_score(y_test, preds),
            "recall":
recall_score(y_test, preds),
            "f1": f1_score(y_test,
preds),
        }
        for k, v in metrics.items():
            print(f"{k}: {v:.4f}")
        return metrics

# -----
-----
---
# Step 4: Select Best Model & Persist
(conceptual deployment prep)
# -----
-----
---
def select_best_model(results: dict):
    best_name = max(results,
key=lambda k: results[k]

```

```

["cv_f1_mean"])
    print(f"Selected best model:
{best_name}")
    return best_name,
results[best_name]["model"]

def save_model(model, path: str =
"churn_model.joblib"):
    """Persist the trained model
artifact for later deployment."""
    joblib.dump(model, path)
    print(f"Model saved to {path}")

# -----
-----
---
# Orchestration
# -----
-----
---
def run_execution_pipeline(X, y,
test_size: float = 0.2):
    X_train, X_test, y_train, y_test
= train_test_split(
        X, y, test_size=test_size,
random_state=42, stratify=y
    )

```

```
        candidates =
get_candidate_models()
        results =
train_and_compare(candidates,
X_train, y_train)
        best_name, best_model =
select_best_model(results)
        metrics =
validate_model(best_model, X_test,
y_test)
        save_model(best_model)
        return best_name, metrics

if __name__ == "__main__":
    df =
pd.read_csv("cleaned_customer_data.csv")
    X = df.drop(columns=["Churn"])
    y = df["Churn"]
    run_execution_pipeline(X, y)
```